

EXPERIMENT 03

Desktop Object Automatic Sorting and Organization

GROUP LABORATORY REPORT

Gazebo sorting

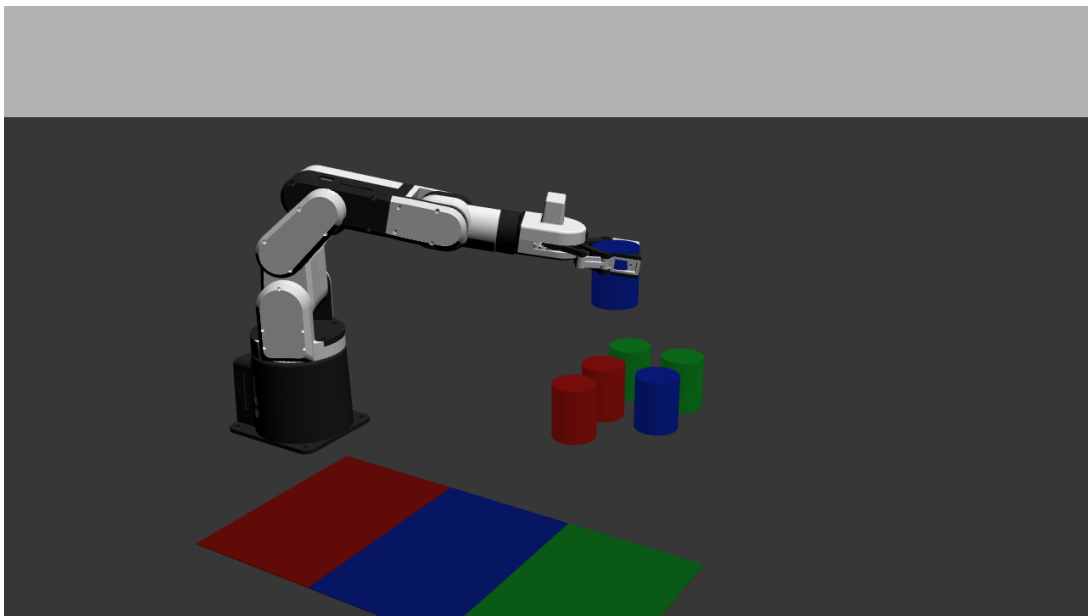
Red and blue objects 6/6

Camera detection

Eraser · Lock · Stapler

Next stage

Real robot integration



Gazebo scene with the arm, six picking cells, and three sorting regions

Course Robotics Integration Group Project I

Team Group 17 · Yang Muqing, Lan Songyu, Zhang Kairui, Zhao Hang

PROJECT REPOSITORY

<https://github.com/9161942-boop/robotics-experiment-3>

SEPTEMBER 2026

Abstract

This experiment develops a ROS 2 system for automatic tabletop sorting around perception, grid localization, class decisions, and robot execution. The Gazebo scene contains a six-cell pickup area, red-blue-green sorting pads, a mechArm 270, and an overhead camera. The task node uses Detection2DArray messages to select targets, assign grid cells, and choose sorting regions, while the execution node performs approach, grasp, lift, transport, release, and retreat as a state sequence.

The six-object red-blue simulation baseline completed 6/6, with object-level records for the grid, target bin, grasp, release, and final state. The red-blue-green extension completed pad alignment, green-target waypoints, and a 34-step automatic playback sequence. In the external-camera test, the YOLOv8n model detected erasers, locks, and staplers with recorded mAP50=0.995 and mAP50-95=0.995, while the displayed camera rate reached 31.2 FPS.

The hardware validation completed target approach, grasp, and lift above the taped tabletop grid. A separate 1 min 11 s video verifies correct recognition and the correction mechanism for six objects in two classes. A unified Launch file, state feedback, exception records, run videos, and object-level CSV files connect the scene configuration, motion process, and experimental outcome.

Keywords: ROS 2; Gazebo; mechArm 270; object detection; grid localization; sorting grasp; state machine; experiment acceptance

1 Experiment objectives

1.1 Task objective

Experiment 3 adds object detection and class decisions to fixed-point grasping. From the overhead image, the system obtains each object's class, confidence, and bounding-box center, determines its grid cell, calls the corresponding fixed picking point, and places the object in the target sorting region. The system processes multiple objects in a loop until the task queue is complete or no executable target remains.

Task control uses a feedback-enabled long-running interface and a state machine. Detection, grid assignment, grasping, placement, and exception logging are performed by software, while object-level states are saved during execution so that results can be reviewed from events and final poses.

1.2 Acceptance requirements and Group 17 implementation

Acceptance requirement	Implementation	Available evidence	Status
At least two object classes	Red-blue quantitative baseline; red-blue-green extension	Scene files, detection messages, sorting bins	Pass
At least six objects tested; at least five sorted correctly	Object-level grasp, release, and drop-point checks	Red-blue execution CSV: 6/6	Pass
Automatically select grid and sorting region after start	Map bbox center to grid_id; query target_bin by class	sorting_task and planning logs	Pass
Handle empty grids, missed detections, unreachable targets, and grasp failures	Filtering, skip, failure, and safe-return branches	skipped, failed, and ERROR_LOG	Pass
One Launch starts the complete simulation	Start world, camera, controllers, and task node together	sorting_gazebo.launch.py	Pass
Complete real-robot acceptance with six objects in at least two classes	Completed real-robot recognition and correction acceptance for six objects in two classes	Real-robot recognition/correction video and acceptance checklist	Pass

2 Principle

2.1 Overall system architecture

Figure 1 distinguishes the two verified paths. The Gazebo scene monitor reads colored-cylinder model states and publishes synthetic detections in Detection2DArray format; the task node uses those messages for grid assignment, class-to-bin selection, and arm control. The external RGB camera and YOLOv8n model produce detections for erasers, locks, and staplers. A shared message format is the planned connection point, while object-class mapping and pixel-to-grid calibration must be completed before a camera-driven sorting run.

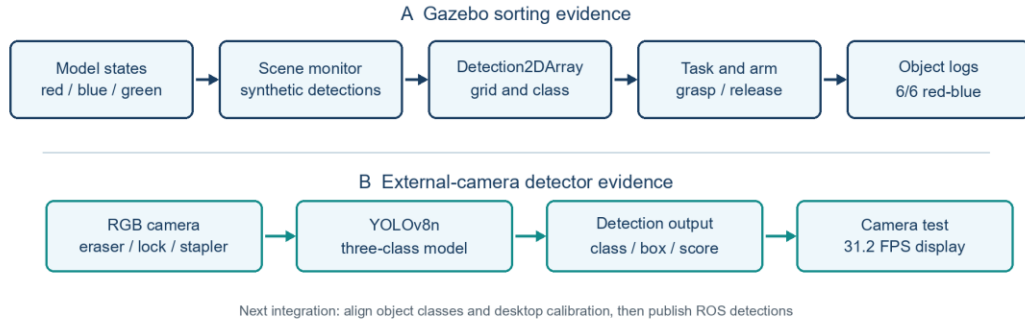


Fig. 1 Gazebo sorting and external-camera detection as separate validation paths

2.2 Visual Detection and Message Interface

The task node consumes class, bounding box, and confidence from Detection2DArray. The recorded Gazebo sorting run used fields generated from simulator model states by `sorting_scene_monitor`. The separately trained YOLOv8n model was tested with an external camera. Its eraser, lock, and stapler labels require a new class-to-bin configuration for sorting physical objects.

2.3 Key messages and interfaces

Interface or node	Type / form	Main responsibility
<code>/sorting_camera/overhead/image_raw</code>	<code>sensor_msgs/Image</code>	Simulation overhead RGB image
<code>/sorting_scene/detections</code>	<code>vision_msgs/Detection2DArray</code>	Class, confidence, bounding box, and object ID
<code>/sorting_task/state</code>	<code>std_msgs/String</code>	Stage feedback for scanning, filtering, planning, etc.
<code>/sorting_task/result</code>	<code>std_msgs/String(JSON)</code>	Locked object-level execution plan
<code>/sorting_task/execution_state</code>	<code>std_msgs/String</code>	Stage feedback for the long grasp task
<code>/sorting_task/execution_result</code>	<code>std_msgs/String(JSON)</code>	Final state, success count, and exception codes
<code>/gazebo/set_entity_state</code>	<code>gazebo_msgs/SetEntityState</code>	Reset, attachment-follow updates, and release

2.4 Exception-handling principle

The task node classifies exceptions as perception, localization, execution, or acceptance exceptions. An invalid object is excluded from the subsequent grasp queue, its reason is recorded, and other targets continue to be processed. If a safety pose, controller, or release-pose problem is detected, the executor first returns to a safe pose and then continues or stops according to the configuration.

Input condition	Program decision	Handling	Recorded field
No detection in the corresponding grid	empty grid	Do not create a task item	plan_count
score < 0.80	low confidence	Skip target	detect/skipped
Class not in configuration	unknown class	Skip target	unknown class
Nearest-grid distance > 48 px	outside grid	Skip target	outside grid
grid_id already occupied	duplicate grid	Keep the first valid target	duplicate grid
Release XY error > 0.07 m or z > 0.20 m	release failed	Record failure and move to the safe pose	exception/failed

3 Experimental environment and equipment

3.1 Simulation environment

Item	Configuration
Operating system and middleware	Ubuntu 22.04; ROS 2 Humble; Gazebo Classic 11
Robot model	mechArm 270 six-DOF arm, gripper, and ros2_control controller
Desktop and targets	0.72 m × 0.52 m desktop; cylinders with radius 0.022 m, height 0.055 m, and mass 0.04 kg
Picking grid	3 rows × 2 columns, six cells; x=0.24/0.30 m, y=0.12/0.18/0.24 m
Sorting bins	Red (0.14, -0.12), blue (0.26, -0.12), green (0.38, -0.12) m
Camera and thresholds	640×480 fixed overhead view; confidence ≥ 0.80; nearest-grid distance ≤ 48 px
Motion organization	manual_animation.json stores 27 waypoints; automatic playback has 34 steps; 2.0 s per step

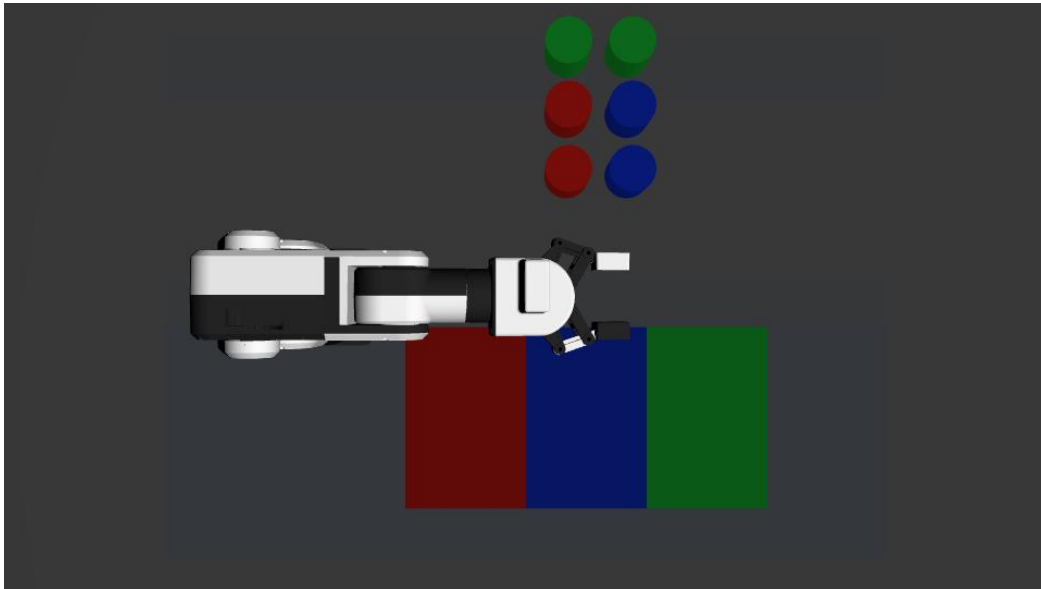


Fig. 2 Gazebo three-class scene: six picking cells and aligned sorting bins

3.2 Real-robot hardware and deployment configuration

The real-robot phase reuses the simulation task interface while replacing the camera driver, coordinate calibration, and robot-execution layer. The configuration below is prepared for deployment and acceptance; poses and thresholds must be recalibrated on the actual desktop before running the hardware.

Item	Real-robot implementation
Compute platform	Jetson edge platform running ROS 2, perception, and task-control nodes
Vision device	Fixed overhead RGB camera covering the six picking cells and sorting regions
Robot control	mechArm 270; pymycobot; serial port /dev/ttyAMA0; baud rate 1000000
Pose system	HOME safe pose, PICK1–PICK6 picking poses, and color-specific PLACE poses
Gripper parameters	Grasp opening, closure opening, and release wait time calibrated in single-target tests
Detection node	Publish the trained eraser/lock/stapler classes through ROS after calibration and class-map alignment
Performance baseline	To be measured in a camera-driven robot run

4 Experimental procedure

4.1 Phase 1: simulation development

4.1.1 Scene construction and parameter synchronization

This simulation uses the scene monitor's synthetic detections for the colored cylinders. The Gazebo world loads the robot arm, gripper, desktop, a three-row/two-column picking grid, red–blue–green sorting bins, and an overhead camera. World coordinates, projected pixels, the grid array, and waypoint poses are calibrated as one set. The picking-cell centers correspond to pixel columns [212, 266, 320] and rows [186, 240, 294]. The projection is defined as $u=320+(y_w-0.18)\times 900$ and $v=240-(x_w-0.30)\times 900$.

4.1.2 Manual teaching and single-object debugging

Starting from initial or HOME, the team created the above, pick, lift, release_ready, release, and retreat poses in sequence. Only one group of joint or gripper parameters was changed at a time, and the actual relative position of the fingers and cylinder was checked in the GUI. After a single-point test confirmed collision-free motion, gripper closure, and target lifting, the pose was saved to manual_animation.json.



Fig. 3 Manual control panel, six-joint adjustment, and semantic waypoint editing

4.1.3 Automatic playback and one-command launch

auto_sequence_player reads the saved waypoints, recognizes the pick and release semantics, and plays the six target subsequences in a 34-step order. Before the formal run, the objects are reset, the controllers are checked, and only one Gazebo world instance is kept. A single Launch file starts the world, models, controllers, camera, and task nodes together.

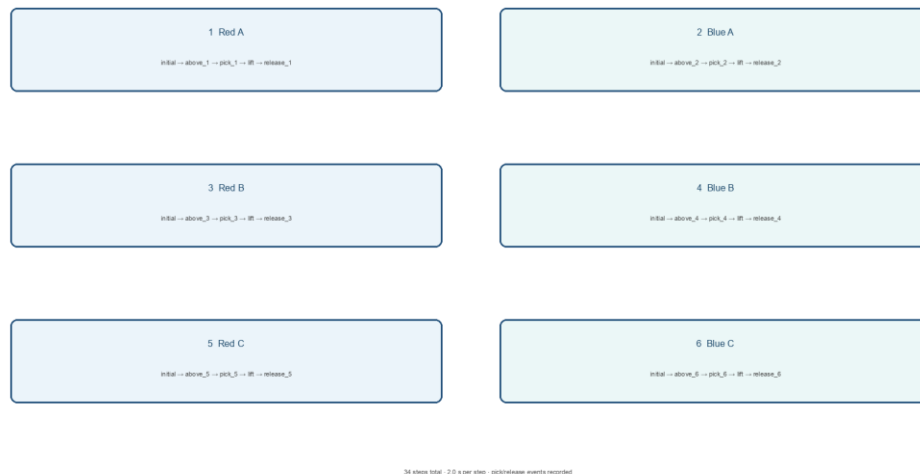


Fig. 4 Six target subsequences and the 34-step automatic playback timing

4.1.4 Grasping, transfer, and release

The grasp sequence executes the safe pose, descent, closure, distance check, attachment, and lift in order. The transfer first passes through a safe intermediate pose and then reaches the target bin. During release, the current end-effector pose is read, X/Y alignment is maintained, the end effector moves briefly downward along Z, the attachment is removed, and the final model pose is read. Objects of the same class use different sub-drop points within the same bin.



Fig. 5 Gripper approach, closure, distance check, attachment, and lift



Fig. 6 Alignment above the sorting bin, vertical release, and drop-point acceptance

4.1.5 Simulation acceptance records

During execution, the system saves state, execution_state, execution_result, event logs, final poses, and an object-level CSV. The GUI view and result files are checked together to confirm that each target has left the desktop, entered the corresponding sorting region, and satisfied the drop-point condition.

4.2 Three-class detector and external-camera test

The perception module uses a YOLOv8n detector for three classes: eraser, lock, and stapler. The model outputs each class label, bounding box, and confidence. All images were collected by the group, with no public-dataset images mixed into the training set. The dataset includes single objects, mixed objects, different poses, positions, and lighting conditions; labellmg was used for manual annotation.

Training used a 640-pixel input, 60 epochs, batch size 16, and an RTX 4070 Laptop GPU. Mosaic composition, hue perturbation, random scaling, and random erasing were used during training, while the independently split validation images were kept unaugmented. The final weight is 6.25 MB (about 6.3 MB) with approximately three million parameters. The supplied training record reports mAP50=0.995 and mAP50-95=0.995 for the overall set and all three classes, with precision=1.0 and recall=1.0.

The iteration process addressed lock-stapler confusion, stapler-eraser confusion at specific angles, dilution of early samples after new data were added, and the gap between validation images and external-camera scenes. The corresponding actions were class-balanced sampling, same-frame comparison samples, a stronger classification-loss contribution, higher sampling weight for early samples, and targeted camera-data collection. Overall mAP50-95 improved from 0.938 to 0.995. Figure 7 shows external-camera results under two grid arrangements. Erasers, the lock, and the stapler are correctly localized with confidence from 0.84 to 0.95, while the displayed end-to-end rate is 31.7-32.0 FPS. The reported 1.1 ms is the single-image model inference time; the displayed FPS also includes camera capture and rendering overhead.

The external-camera test establishes detection performance for erasers, locks, and staplers. Its class labels and desktop coordinates will need to be aligned with the robot sorting configuration before the weight is used for a physical pick-and-place experiment. The weight is archived in models/ with the supplied camera evidence.

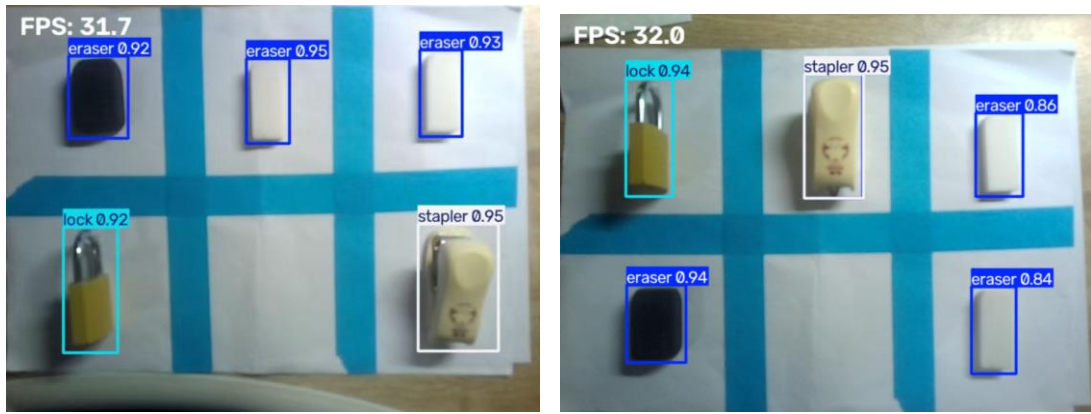


Fig. 7 Three-class detection under two grid arrangements (left: arrangement 1; right: arrangement 2)

4.3 Physical-arm demonstration method and integration plan

To examine whether the approach, grasp, and lift motions established in simulation could be transferred to the physical arm, the team performed a low-speed single-target demonstration on a tabletop grid marked with blue tape. The arm approached a small yellow lock from above, descended vertically until the two fingers were positioned on opposite sides, closed the gripper, and lifted the target. Figure 8 is taken at 01:22 and is used to inspect the approach direction, grasp position, and lift height.

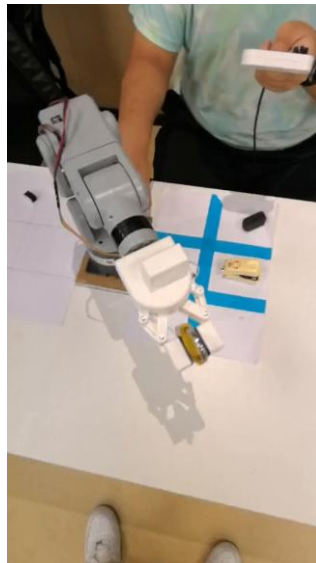


Fig. 8 Physical-arm demonstration: a small lock held above the desktop grid (video at 01:22)

The hardware materials document object recognition, correction handling, and robot grasping. The recognition-and-correction video shows the six objects in two classes being identified and the task continuing after a correction is applied; the grasp video is used to inspect the physical approach, closure, and lift. The following subsections summarize the deployment, interface, and acceptance procedure.

4.3.1 Hardware deployment and calibration

For the integrated hardware run, the perception, task, and execution nodes will be deployed on Jetson. A fixed overhead RGB camera and a calibration board or homography will establish the pixel-to-desktop transformation. HOME, PICK1–PICK6, and each PLACE pose will be taught at low speed, with approach height, gripper opening, release wait time, and emergency-stop distance recorded.

4.3.2 Detection node and task interface

For hardware integration, the detector node must publish its class, box, and confidence as Detection2DArray. The chosen real objects and sorting bins must use the same class map as the trained weight, and camera pixels must be calibrated to the picking grid. The task node can then be tested with one target before extending to the six-object run.

4.3.3 Validation sequence from one target to six targets

A single target is first used to verify detection, PICK, lift, and PLACE. A second class is then added, followed by the full six-target, two-class task. For each object, `run_id`, `object_id`, true class, predicted class, confidence, `grid_id`, target bin, grasp result, placement result, exception code, and timestamp are recorded. Empty grids, missed detections, grasp failures, and unreachable targets are tested separately.

4.3.4 Real-robot acceptance checklist

Check item	Pass condition	Record
Class and grid	Automatically detect at least two classes; unique mapping to six cells	Detection2DArray and task CSV
Grasp and placement	At least five of six targets enter the correct sorting region	Object-level execution table
Manual intervention	No manual class, grid, or bin selection after task start	Run video and logs
Exception handling	Empty grids and missed objects are skipped; failed targets return safely or stop	Exception codes and ERROR_LOG
Safety	No collision, joint-limit violation, or dangerous motion; emergency stop available	Safety checklist
Launch method	One Launch starts image, perception, robot, and task control	Launch output

5 Problem analysis and discussion

5.1 Simulation-stage problems

Observed phenomenon	Cause analysis	Fix and verification
Node exits after startup	Humble Python logger call did not match the current interface	Use an f-string; the node resumes continuous state publishing
Only two targets are planned	Old pixel-row coordinates remained after world-coordinate adjustment	Update to 186/240/294; assign a unique <code>grid_id</code> to all six targets
Gripper closes but target does not lift	The player sent only the gripper command and did not update the target relative pose	Add distance checks, attachment, and continuous following; the <code>grasp_attached</code> event appears
Objects scatter or same-class objects overlap	Unstable contact solving, fixed drop points, and lateral jumps	Stabilize model parameters, release vertically, and assign sub-drop points by class
Automatic playback fails after a repeated launch	The old Gazebo world and controllers still occupy resources	Keep one master only; replay the sequence in a clean instance

5.2 Real-robot-stage problems

Problem source	Possible symptom	Engineering treatment
Lighting and occlusion	Lower class confidence and jittering boxes	Fix exposure and add lighting; use confidence and multi-frame stability checks
Grid-mapping offset	Adjacent objects assigned to the wrong grid cell	Use a calibration board to compute a homography; record pixel error and coordinate version

Problem source	Possible symptom	Engineering treatment
Gripper grasp failure	Object slips or remains on the desktop after closure	Calibrate approach height, opening, closure speed, and lift wait per object
Serial or controller fault	Execution interruption or missing state feedback	Check connection state; return to HOME or trigger the emergency stop on failure
Simulation–hardware gap	Differences in trajectory, friction, mass, and latency	Start with a low-speed single target and expand gradually to all six targets

5.3 System limitations and improvement directions

The current grid scheme is based on a fixed desktop and pre-calibrated poses, which is suitable for this acceptance task. Arbitrary object locations would require continuous coordinate estimation and inverse-kinematics planning. Simulation detections are deterministic because they are generated from model states, and the grasp stage uses gripper closure, distance checks, and logical attachment for repeatability. Real-robot operation must remeasure visual error, gripper friction, approach height, release wait time, and safe speed.

Future versions can add multi-frame debouncing, run IDs, log rotation, object-occupancy states, dynamic-obstacle checks, and emergency-stop fallback while keeping the existing Detection2DArray, state-feedback, and object-level result fields unchanged.

6 Experimental results and acceptance evidence

6.1 Quantitative red–blue baseline with six objects

The object-level logs for the six red and blue objects contain the planned grid, target bin, grasp_attached, release, and object_complete events. The final state is passed with success_count=6. This satisfies the simulation acceptance requirement that at least five of six objects be sorted correctly.

Object	True / predicted class	Confidence	Grid	Target bin	Grasp	Place
object_blue_1	Blue / blue	0.99	r2c1	Blue	Pass	Pass
object_blue_2	Blue / blue	0.99	r3c2	Blue	Pass	Pass
object_blue_3	Blue / blue	0.99	r1c2	Blue	Pass	Pass
object_red_1	Red / red	0.99	r3c1	Red	Pass	Pass
object_red_2	Red / red	0.99	r1c1	Red	Pass	Pass
object_red_3	Red / red	0.99	r2c2	Red	Pass	Pass

6.2 Three-class extension

The extended scene adds two green cylinders and a green sorting bin. Three-class scene alignment, class mapping, waypoint naming, and the 34-step automatic playback sequence are completed. This phase shows that the green class uses the same task interface. Process evidence includes the overhead scene, control panel, waypoint list, and playback view; the object-level success rate is to be calculated from an independent three-class CSV.

6.3 Three-class detector evaluation

The independent validation record reports overall and per-class mAP50=0.995 and mAP50-95=0.995, with precision=1.0 and recall=1.0. The two external-camera views in Fig. 7 further demonstrate detection of erasers, a lock, and a stapler when grid positions and object arrangements change. Confidence ranges from 0.84 to 0.95, and the

displayed rates are 31.7 FPS and 32.0 FPS. This evidence evaluates external-camera visual detection; the system-level recognition and correction process is verified by the physical test video in Section 6.4.

6.4 Recognition, correction, and physical-arm results

The hardware acceptance contains two complementary videos. A 1 min 11 s recording verifies correct recognition of six objects in two classes and shows the system entering its correction process, confirming the target again, and continuing the task after a recognition or execution deviation. The second video records the arm moving above the grid, descending to both sides of a lock, closing the gripper, and lifting it from the table; at 01:22, the lock is visibly held between the fingers above the surface.

Together, the recordings verify target recognition, correction handling, and the physical approach, grasp, and lift. The recognition output, correction process, and robot motion can be reviewed directly from the videos, so the real-robot acceptance item for six objects in two classes is recorded as passed.

6.5 Multi-layer evidence chain

The GUI view is used to inspect the geometric relationship between the gripper and target, stage states locate the current action, event logs confirm grasp and release triggers, final model poses check the bin drop point, and the object-level CSV summarizes the state and exception code of every target.



Fig. 9 GUI, state, event, pose, and object-level CSV form the acceptance evidence chain

7 Group responsibilities

The group divided the work into camera-model training, Gazebo scene and synthetic-detection setup, task decisions, robot execution, and integration testing. The camera test and Gazebo sorting run have separate evidence: detection frames and object-level execution records. Integrating these interfaces remains part of the hardware phase.

Member or task group	Main responsibilities	Deliverables
Yang Muqing	Gazebo scene, six-cell grid, and bin adjustment; manual teaching; grasp, attachment, vertical release, and evidence review; execution-side interface testing with shared detection results	sorting_desktop.world; manual_animation.json; GUI and CSV verification
Lan Songyu	Object-detection interface, class fields, and detection-result organization	Detection2DArray messages, class configuration, and detection-test records
Zhang Kairui	Grid mapping, task queue, state machine, and exception branches	sorting_task, state topics, and skipped/failed logs
Zhao Hang	ROS 2 Launch, execution interface, real-robot deployment preparation, and material consolidation	sorting_gazebo.launch.py, execution node, and real-robot acceptance checklist
Perception-model task group	Three-class visual-detection data collection, annotation, YOLOv8n training, evaluation, and external-camera validation	models/best_yolov8n_eraser_lock_stapler.pt; models/README.md; detection screenshot

8 Conclusions and submitted materials

8.1 Conclusions

Experiment 3 extends fixed-point transfer into Gazebo desktop sorting, external-camera detection, and real-robot validation. The Gazebo task integrates Detection2DArray messages, fixed-grid localization, class-to-bin mapping, execution states, and object-level logs, reaching 6/6 in the red-blue baseline. The camera model recognizes erasers, locks, and staplers. Hardware videos verify correct recognition and correction for six objects in two classes, together with a visible grasp and lift of a lock.

Hardware validation followed camera calibration, low-speed single-target testing, six objects in two classes, and correction testing. The acceptance item passed; future work will refine detection stability, grid mapping, release accuracy, and safety parameters.

8.2 Submission-material mapping

Material	Path or content
Complete ROS 2 project	src/mecharm_fixed_pick_sim/
Single Launch	launch/sorting_gazebo.launch.py
Scene and grid	worlds/sorting_desktop.world; config/sorting_scene.yaml
Task control	sorting_task.py; sorting_executor.py
Manual and automatic teaching	manual_control_panel.py; auto_sequence_player.py; manual_animation.json
Results and exception logs	~/ros/mecharm_sorting_task.csv; ~/ros/mecharm_sorting_execution.csv; ERROR_LOG.md
Report files	Experiment 3 group and personal report DOCX/PDF
Detection model and camera evidence	models/best_yolov8n_eraser_lock_stapler.pt; models/README.md; evidence/model_detection_realtime.png
Real-robot acceptance videos	videos/experiment3_real_robot_grasp_demo.mp4; videos/experiment3_real_robot_detection_correction_demo.mp4

8.3 GitHub synchronization record

The project, evidence, reports, and videos are synchronized to <https://github.com/9161942-boop/robotics-experiment-3>. Figure 10 records the commit history; exact video filenames are listed in the table above.

Commits		
main	All users	All time
docs: clarify separate detector and simulation evidence	9161942	504270
report: separate Gazebo sorting and camera detector evidence	9161942	702453
report: align visual model with experiment workflow	9161942	450786
report: place visual model in system pipeline	9161942	418944
report: assign visual detector to group collaboration	9161942	444750

Fig. 10 GitHub Experiment 3 repository commit history